



Asignatura:

Programación Paralela

Título del proyecto:

Filtro de imágenes

Integrantes:

Richard Alexander Torres Pérez

Ismarlin Martínez Mejia

Líder de equipo:

Rusbel Ohara Uceta Sosa

Fecha de entrega:

19/8/2026

Índice

Índice.....	2
1. Introducción	4
1.1. Presentación General del Proyecto	4
1.2. Justificación del tema elegido	5
1.3. Objetivos	7
1.3.1. Objetivo General	7
1.3.2. Objetivos Específicos	7
2. Descripción del Problema.....	9
3. Cumplimiento de los Requisitos del Proyecto	11
4. Diseño de la Solución	13
4.1 Arquitectura general del sistema	13
4.2 Diagrama de componentes y tareas paralelas.....	15
4.3 Estrategia de paralelización utilizadas	17
4.4 Herramientas y tecnologías empleadas	18
5. Implementación Técnica.....	19
5.1. Descripción de la estructura del proyecto:	19
5.2. Explicación del código clave:	21
5.3. Uso de mecanismos de sincronización:.....	22
5.4. Justificación técnica de las decisiones tomadas:	24
6. Evaluación de Desempeño	26
6.1. Metodología de la comparativa	26
6.2. Comparativa entre ejecución secuencial y paralela.....	26
6.3. Métricas: tiempo de ejecución, eficiencia, escalabilidad	27
6.4. Gráficas con resultados	29
6.5. Análisis de cuellos de botella o limitaciones.....	30
7. Trabajo en equipo:	31
7.1. Descripción del reparto de tareas:	31
7.2. Herramientas utilizadas para coordinación:	31
8. Conclusiones.....	32
Principales aprendizajes técnicos:.....	32
Retos enfrentados y superados	32
Posibles mejoras o líneas futuras	33
9.Referencias	33
Fuentes bibliográficas	34

10. Anexos	35
10.1 Manual de ejecución del sistema	35
10.1.1 Descarga y apertura del proyecto	35
10.1.2 Compilación inicial y creación de directorios	36
10.1.3 Preparación de las imágenes	37
10.1.4 Ejecución de la aplicación	38
10.1.5 Procesamiento y presentación de resultados	39
10.1.6 Verificación de las imágenes generadas	40
10.2 Capturas adicionales y pruebas complementarias.....	41
10.2.1 Prueba con 500 imágenes	41
10.2.2 Comparación de las estrategias	42
10.2.3 Validaciones realizadas.....	44
10.3 Enlace al repositorio público de Git	44

1. Introducción

1.1. Presentación General del Proyecto

El presente proyecto tiene como propósito desarrollar una aplicación de consola utilizando el lenguaje de programación C#, cuya función principal será el procesamiento digital de imágenes mediante la aplicación de diversos filtros gráficos. Entre los filtros que se implementarán se encuentran la escala de grises, la inversión de colores, el ajuste de brillo, el contraste y el desenfoque, permitiendo modificar la apariencia de una imagen de manera automática y eficiente.

El objetivo principal del proyecto es demostrar las ventajas que ofrece la programación paralela en tareas de procesamiento intensivo, aprovechando la capacidad de los procesadores multinúcleo para ejecutar múltiples operaciones de forma simultánea. El paralelismo se aplicará en dos niveles complementarios: por un lado, por un lado, cada imagen podrá dividirse en distintas regiones, asignando cada una a una tarea independiente ejecutada en paralelo; por otro lado, al procesar un conjunto de imágenes, cada una podrá asignarse a una tarea distinta para procesarlas de forma simultánea. Una vez que todas las tareas finalicen, los resultados se combinarán para obtener la imagen completamente procesada.

Para la implementación del paralelismo se empleará una estrategia recursiva de división de tareas, donde la imagen será fragmentada progresivamente hasta alcanzar un tamaño adecuado para su procesamiento. Este enfoque permitirá distribuir la carga de trabajo de forma equilibrada entre los diferentes núcleos del procesador, reduciendo considerablemente el tiempo de ejecución en comparación con un procesamiento secuencial tradicional.

La aplicación estará diseñada con una arquitectura modular, facilitando la incorporación de nuevos filtros o algoritmos de procesamiento en futuras versiones. Además, el usuario podrá seleccionar desde la consola el filtro que desea aplicar, indicando la imagen de entrada y obteniendo como resultado una nueva imagen con las modificaciones correspondientes.

Como parte del desarrollo, también se realizarán pruebas comparativas entre la ejecución secuencial y la ejecución paralela, registrando los tiempos de procesamiento obtenidos para imágenes de distintos tamaños y resoluciones. Estos resultados permitirán analizar

el impacto del paralelismo sobre el rendimiento de la aplicación, evidenciando las mejoras alcanzadas cuando se distribuye el trabajo entre múltiples tareas concurrentes.

En conjunto, este proyecto busca integrar conceptos fundamentales de programación paralela, recursividad, procesamiento digital de imágenes y optimización del rendimiento, demostrando cómo estas técnicas pueden combinarse para desarrollar aplicaciones más rápidas, eficientes y escalables, especialmente en escenarios donde el procesamiento de grandes volúmenes de datos representa un desafío computacional.

1.2. Justificación del tema elegido

1) Ejecución simultánea de múltiples tareas

Este proyecto cumple con el principio de ejecución simultánea de múltiples tareas debido a que varias imágenes pueden dividirse en diferentes regiones, bloques o segmentos independientes. Cada uno de estos fragmentos puede ser procesado de forma paralela mediante tareas separadas, permitiendo que varios núcleos del procesador trabajen al mismo tiempo. Como resultado, el procesamiento de filtros como escala de grises, inversión de colores, brillo, contraste o desenfoque se realiza de manera más rápida que en un enfoque secuencial, donde cada píxel debe procesarse uno tras otro. Esta característica convierte al proyecto en un excelente ejemplo de aplicación de la programación paralela para resolver problemas de alto costo computacional.

2) Necesidad de compartir datos entre tareas

El proyecto también requiere el intercambio y acceso compartido a diferentes estructuras de datos durante su ejecución. Todas las tareas deben leer información de la imagen original y escribir los resultados en una imagen de salida o en estructuras de almacenamiento compartidas. Asimismo, es posible que varias tareas necesiten actualizar contadores relacionados con el progreso del procesamiento, el número de píxeles procesados o estadísticas de rendimiento. Debido a que múltiples hilos pueden acceder simultáneamente a estos recursos, es necesario implementar mecanismos de sincronización como lock, Interlocked u otras técnicas de control de concurrencia,

garantizando la integridad de los datos y evitando condiciones de carrera que puedan afectar el resultado final.

3) Exploración de diferentes estrategias de paralelización

Uno de los principales objetivos del proyecto es analizar el comportamiento de distintas técnicas de paralelización para determinar cuál ofrece un mejor rendimiento dependiendo del tamaño de la imagen y del tipo de filtro aplicado. Para ello, se podrán comparar enfoques como el procesamiento completamente secuencial, la división por filas, la división por bloques, la utilización de Task.Run, el uso de Parallel.For y una estrategia recursiva de división de tareas que distribuya automáticamente la carga de trabajo. La comparación entre estas alternativas permitirá comprender las ventajas, limitaciones y escenarios de aplicación de cada técnica, fortaleciendo el aprendizaje sobre programación paralela y optimización de algoritmos.

4) Capacidad de escalar con más recursos

La naturaleza del procesamiento paralelo permite que el rendimiento de la aplicación mejore conforme aumenta la capacidad del hardware disponible. Al ejecutarse en equipos con procesadores que poseen un mayor número de núcleos o hilos de ejecución, será posible distribuir el procesamiento de una mayor cantidad de segmentos de la imagen de forma simultánea. Esto significa que, a medida que los recursos computacionales disponibles crecen, el tiempo total requerido para aplicar los filtros disminuye considerablemente. Esta propiedad de escalabilidad demuestra cómo una aplicación correctamente diseñada puede aprovechar las mejoras del hardware moderno sin necesidad de modificar significativamente su lógica principal.

5) Métricas de evaluación del rendimiento

El proyecto permitirá obtener y analizar diferentes métricas que servirán para evaluar objetivamente la eficiencia de la solución implementada. Entre las principales mediciones se encuentran el tiempo de ejecución utilizando procesamiento secuencial, el tiempo

obtenido mediante procesamiento paralelo, el Speedup alcanzado al comparar ambos enfoques, la eficiencia del paralelismo en función del número de núcleos utilizados y la cantidad de píxeles procesados por segundo. Estas métricas facilitarán la elaboración de tablas, gráficos y análisis comparativos que evidencien el impacto positivo del paralelismo sobre el rendimiento de la aplicación.

6) Aplicación a escenarios del mundo real

El procesamiento digital de imágenes constituye una tecnología ampliamente utilizada en numerosos sectores de la industria y la investigación, lo que convierte este proyecto en una aplicación con un contexto completamente realista. Técnicas similares son empleadas en programas de edición fotográfica, aplicaciones móviles de cámaras, plataformas de redes sociales, sistemas de visión artificial, reconocimiento facial, análisis de imágenes médicas, inspección industrial, vehículos autónomos y sistemas inteligentes de vigilancia y seguridad. Debido a que estos sistemas suelen procesar grandes volúmenes de imágenes en tiempos reducidos, el uso de programación paralela representa una solución eficiente para mejorar el rendimiento y optimizar el aprovechamiento de los recursos de hardware disponibles.

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar una aplicación de consola en C# capaz de procesar imágenes mediante la aplicación de diferentes filtros digitales, implementando técnicas de programación paralela y una estrategia recursiva de división de tareas para optimizar el tiempo de procesamiento y evaluar el impacto del paralelismo en el rendimiento de la aplicación.

1.3.2. Objetivos Específicos

- Diseñar e implementar una aplicación de consola que permita cargar una imagen, aplicar distintos filtros de procesamiento digital y generar una nueva imagen con los cambios realizados.

- Implementar filtros de procesamiento de imágenes, tales como escala de grises, inversión de colores, ajuste de brillo, contraste y desenfoque, garantizando resultados correctos y consistentes.
- Aplicar una estrategia recursiva para dividir la imagen en múltiples segmentos independientes, permitiendo distribuir el trabajo entre varias tareas ejecutadas en paralelo.
- Utilizar las herramientas de programación paralela proporcionadas por C#, como Task, Parallel.For o mecanismos equivalentes, para ejecutar simultáneamente el procesamiento de las diferentes regiones de la imagen.
- Implementar mecanismos de sincronización cuando sea necesario para garantizar la integridad de los datos compartidos y evitar conflictos durante la ejecución concurrente.
- Comparar el rendimiento del procesamiento secuencial con el procesamiento paralelo mediante la medición de tiempos de ejecución y otras métricas de desempeño.
- Calcular indicadores de rendimiento como el tiempo de procesamiento, Speedup, eficiencia y rendimiento en píxeles procesados por segundo, con el fin de analizar los beneficios obtenidos mediante el paralelismo.
- Evaluar el comportamiento de la aplicación utilizando imágenes de diferentes tamaños y resoluciones para determinar cómo influye el volumen de datos en el desempeño del algoritmo.
- Diseñar una arquitectura modular que facilite la incorporación de nuevos filtros o algoritmos de procesamiento sin modificar significativamente la estructura principal de la aplicación.
- Demostrar la aplicabilidad de la programación paralela en problemas reales de procesamiento digital de imágenes, evidenciando cómo el aprovechamiento de procesadores multinúcleo contribuye a reducir los tiempos de ejecución y mejorar la eficiencia computacional.

2. Descripción del Problema

1) Contexto del problema seleccionado

El procesamiento digital de imágenes es una de las tareas computacionales más exigentes de la informática moderna, debido a la cantidad de operaciones que deben aplicarse sobre cada píxel de una imagen. Una fotografía estándar puede contener millones de píxeles, y cada uno debe leerse, evaluarse y transformarse al aplicar un filtro. Cuando esto se hace de forma secuencial, con un único hilo de ejecución, el tiempo requerido crece considerablemente conforme aumentan el tamaño y la resolución de la imagen.

Este comportamiento representa un problema real, ya que muchas aplicaciones actuales necesitan procesar imágenes en tiempos muy reducidos, incluso en tiempo real. Un procesamiento lento afecta el rendimiento general y limita la capacidad de escalar cuando se trabaja con imágenes de alta resolución, lotes grandes de fotografías o flujos continuos de video.

Por esta razón, el problema seleccionado consiste en optimizar el tiempo de procesamiento de filtros gráficos aplicados sobre imágenes digitales, aprovechando los múltiples núcleos disponibles en el hardware actual. En lugar de un enfoque secuencial que subutiliza el procesador, se plantea dividir el trabajo, ya sea fragmentando una imagen en distintas regiones o distribuyendo varias imágenes entre distintas tareas, para procesarlas de forma simultánea y reducir así el tiempo total necesario.

2) Aplicación del problema en un escenario real

Este tipo de problema aparece constantemente en distintos sectores. Un ejemplo claro son las aplicaciones móviles de edición fotográfica y las redes sociales, donde millones de usuarios suben, procesan y aplican filtros a imágenes de forma simultánea, y las transformaciones deben completarse en milisegundos para no afectar la experiencia del usuario.

Otro caso relevante es la visión artificial y el reconocimiento facial, donde las imágenes deben procesarse rápidamente para detectar patrones u objetos. En sistemas de vigilancia inteligente, el procesamiento de video en tiempo real depende directamente de la velocidad con la que cada fotograma puede analizarse, lo que hace indispensable el uso de técnicas paralelas.

En medicina, el análisis de imágenes diagnósticas (radiografías, tomografías, resonancias) requiere procesar grandes volúmenes de datos con rapidez, ya que el tiempo de respuesta puede influir en decisiones clínicas. En la industria manufacturera, los sistemas de inspección visual automatizada dependen igualmente de procesar imágenes con la mayor velocidad posible durante líneas de producción de alta velocidad.

Estos ejemplos evidencian que el problema abordado no es un caso aislado, sino una necesidad real y recurrente en múltiples industrias.

3) Importancia del paralelismo en la solución

La programación paralela es fundamental para resolver este problema porque permite distribuir una tarea costosa entre varios núcleos, en lugar de ejecutarla en un único hilo. En esta aplicación, el trabajo puede distribuirse de dos maneras: fragmentando una misma imagen en regiones independientes entre sí, o asignando imágenes completas de un lote a distintas tareas. En ambos casos, el procesamiento de una unidad no depende de los resultados obtenidos en otra, lo que hace que el problema sea altamente paralelizable.

Al aplicar paralelismo, el tiempo total de procesamiento se reduce de manera significativa, especialmente con imágenes de gran tamaño o alta resolución. Esto no solo se traduce en una respuesta más rápida, sino en un mejor aprovechamiento del hardware disponible, evitando que los núcleos adicionales permanezcan inactivos. Además, el paralelismo permite que la aplicación escale de forma natural: a mayor número de núcleos, mejor rendimiento, sin necesidad de modificar su lógica interna.

En consecuencia, el paralelismo no es solo una alternativa técnica para acelerar el procesamiento de imágenes, sino un elemento esencial para que la solución sea viable y aplicable a escenarios reales donde el tiempo de respuesta y el volumen de datos son factores críticos.

3. Cumplimiento de los Requisitos del Proyecto

El proyecto cumple con los requisitos establecidos porque propone una aplicación de consola en C# capaz de procesar grandes cantidades de imágenes aplicando filtros digitales de manera paralela. En lugar de procesar cada imagen una por una de forma secuencial, el sistema distribuirá el trabajo entre múltiples tareas para que varias imágenes puedan ser procesadas al mismo tiempo. Esto permitirá demostrar la ventaja del paralelismo, el manejo de datos compartidos, la comparación de estrategias de ejecución, la escalabilidad y la medición del rendimiento.

1) Ejecución simultánea de múltiples tareas

El proyecto cumple con este requisito porque el sistema podrá procesar muchas imágenes de forma simultánea. Cada tarea puede encargarse de una imagen diferente o de un grupo de imágenes dentro de una carpeta. De esta forma, mientras una tarea aplica un filtro a una imagen, otras tareas pueden estar procesando imágenes distintas al mismo tiempo.

Esto permite aprovechar mejor los núcleos del procesador y reducir el tiempo total cuando se trabaja con grandes volúmenes de imágenes. En lugar de esperar a que una imagen termine para iniciar la siguiente, el programa distribuye la carga de trabajo entre varias tareas concurrentes.

2) Necesidad de compartir datos entre tareas

El proyecto requiere compartir datos entre tareas porque todas las tareas que procesan imágenes deben actualizar información común del sistema. Entre estos datos compartidos se encuentran la cantidad total de imágenes procesadas, el progreso general, los errores encontrados, las rutas de salida, las métricas de rendimiento y el conteo de imágenes completadas exitosamente.

Debido a que varias tareas pueden intentar modificar estos datos al mismo tiempo, será necesario usar mecanismos de sincronización como lock, Interlocked o colecciones seguras para concurrencia. Esto permitirá evitar condiciones de carrera y garantizar que los resultados finales sean correctos.

3) Exploración de diferentes estrategias de paralelización

El proyecto permite explorar diferentes estrategias de paralelización porque el procesamiento de imágenes puede abordarse de varias maneras.

Primero, se puede implementar una versión secuencial donde las imágenes se procesan una por una. Luego, se puede comparar con una versión paralela donde varias imágenes se procesan al mismo tiempo usando Task.Run, Parallel.ForEach o una división por grupos de imágenes.

También se puede agregar una estrategia adicional para imágenes de alta resolución, dividiendo una misma imagen en regiones, filas o bloques. De esta forma, el proyecto no solo compara procesamiento secuencial contra paralelo, sino también diferentes formas de distribuir el trabajo.

4) Escalabilidad con más recursos

El proyecto tiene capacidad de escalar porque, mientras más núcleos tenga el procesador, mayor cantidad de imágenes podrán procesarse simultáneamente. Si el programa se ejecuta en una computadora con más recursos, se puede aumentar el grado de paralelismo para distribuir mejor la carga de trabajo.

Esto es especialmente útil cuando se trabaja con cientos o miles de imágenes, ya que el sistema puede aprovechar el hardware disponible para reducir el tiempo total de procesamiento sin cambiar completamente la lógica del programa.

5) Métricas de evaluación del rendimiento

El proyecto permite incluir métricas claras para evaluar el rendimiento de la solución. Se pueden medir el tiempo total de procesamiento secuencial, el tiempo total de procesamiento paralelo, el Speedup, la eficiencia, la cantidad de imágenes procesadas por segundo y la cantidad de píxeles procesados por segundo.

Estas métricas permitirán comparar objetivamente si el procesamiento paralelo ofrece una mejora real frente al procesamiento secuencial. Además, se podrá analizar cómo cambia

el rendimiento al variar la cantidad de imágenes, el tamaño de las imágenes y el número de tareas utilizadas.

6) Aplicación a un problema del mundo real

El proyecto se aplica a un problema real porque muchas aplicaciones actuales necesitan procesar grandes cantidades de imágenes en poco tiempo. Esto ocurre en redes sociales, aplicaciones de cámara, edición fotográfica, sistemas de vigilancia, visión artificial, reconocimiento facial, imágenes médicas y plataformas que manejan archivos multimedia de forma masiva.

En estos escenarios, procesar imágenes una por una puede ser demasiado lento. Por eso, el paralelismo es una solución adecuada para mejorar el rendimiento y permitir que el sistema trabaje con grandes volúmenes de datos visuales de forma más eficiente.

4. Diseño de la Solución

4.1 Arquitectura general del sistema

La aplicación fue diseñada utilizando una arquitectura modular, con el propósito de separar las responsabilidades relacionadas con la interacción del usuario, la aplicación de filtros, el procesamiento de imágenes y el cálculo de métricas. Esta organización facilita el mantenimiento del código y permite incorporar nuevas estrategias de procesamiento sin modificar completamente el sistema.

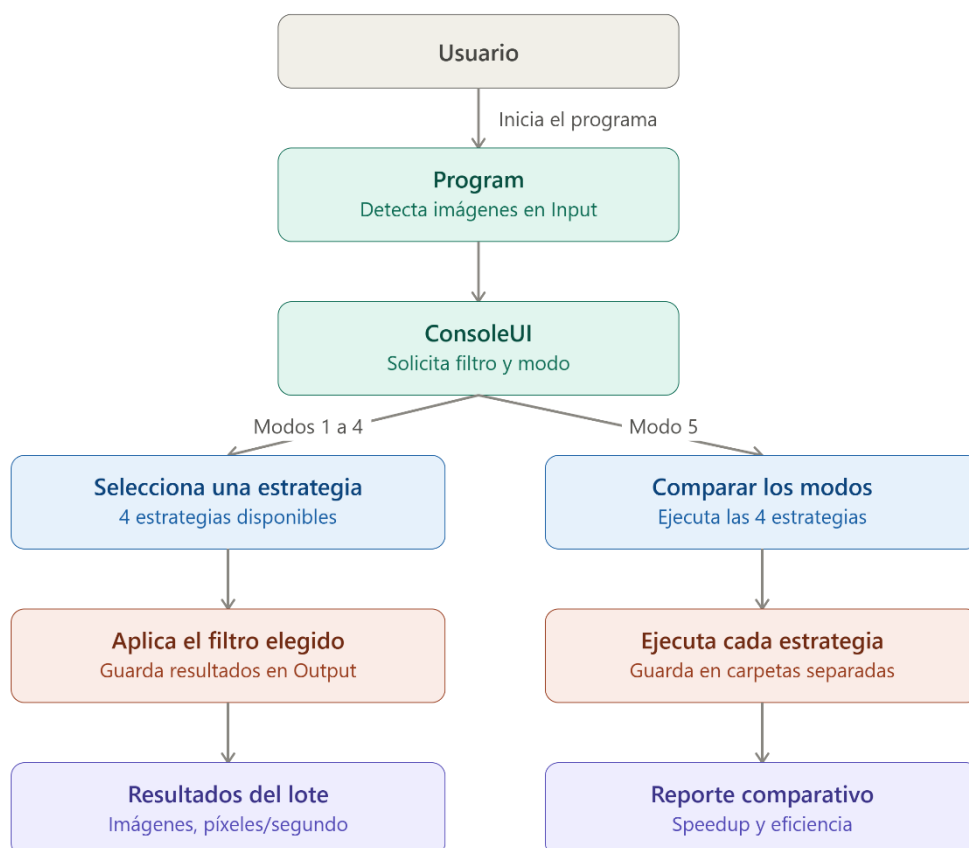
El programa inicia en la clase **Program**, que funciona como punto de entrada y coordina el flujo general de la aplicación. Desde esta clase se muestran las opciones disponibles, se detectan las imágenes almacenadas en el directorio de entrada, se selecciona el filtro que será aplicado y se determina el modo de procesamiento.

La solución se encuentra dividida en los siguientes componentes:

- **ConsoleUI:** contiene el menú de consola y la presentación de los resultados. Se encarga de solicitar al usuario el filtro, el valor requerido por determinados filtros y la estrategia de procesamiento.

- **Models:** contiene las enumeraciones y clases que representan los datos utilizados por el sistema, como los tipos de filtros, los modos de procesamiento, las rutas de la aplicación y los resultados obtenidos.
- **Filters:** contiene la implementación de los filtros de escala de grises, inversión de colores, brillo, contraste y desenfoque. Los filtros pueden aplicarse secuencialmente o sobre regiones específicas de una imagen.
- **Processing:** contiene las diferentes estrategias utilizadas para procesar las imágenes: secuencial, paralela por filas, división recursiva y procesamiento paralelo por lotes.
- **Metrics:** se encarga de calcular el tiempo total, la cantidad de imágenes procesadas, los píxeles procesados por segundo, el speedup y la eficiencia de cada estrategia.

El flujo general de información puede representarse de la siguiente manera:



Las imágenes originales son leídas desde el directorio Input. Después de aplicar el filtro seleccionado, los resultados se guardan en directorios diferentes de acuerdo con la estrategia utilizada, tales como Output/Sequential, Output/Parallel, Output/Recursive y Output/ParallelBatch. Esta separación permite comprobar visualmente los resultados y evita que las estrategias sobrescriban las imágenes generadas por otras modalidades.

4.2 Diagrama de componentes y tareas paralelas

El sistema implementa cuatro modos de procesamiento. El modo secuencial sirve como referencia para medir el rendimiento, mientras que los otros tres modos distribuyen el trabajo utilizando diferentes técnicas de paralelismo.

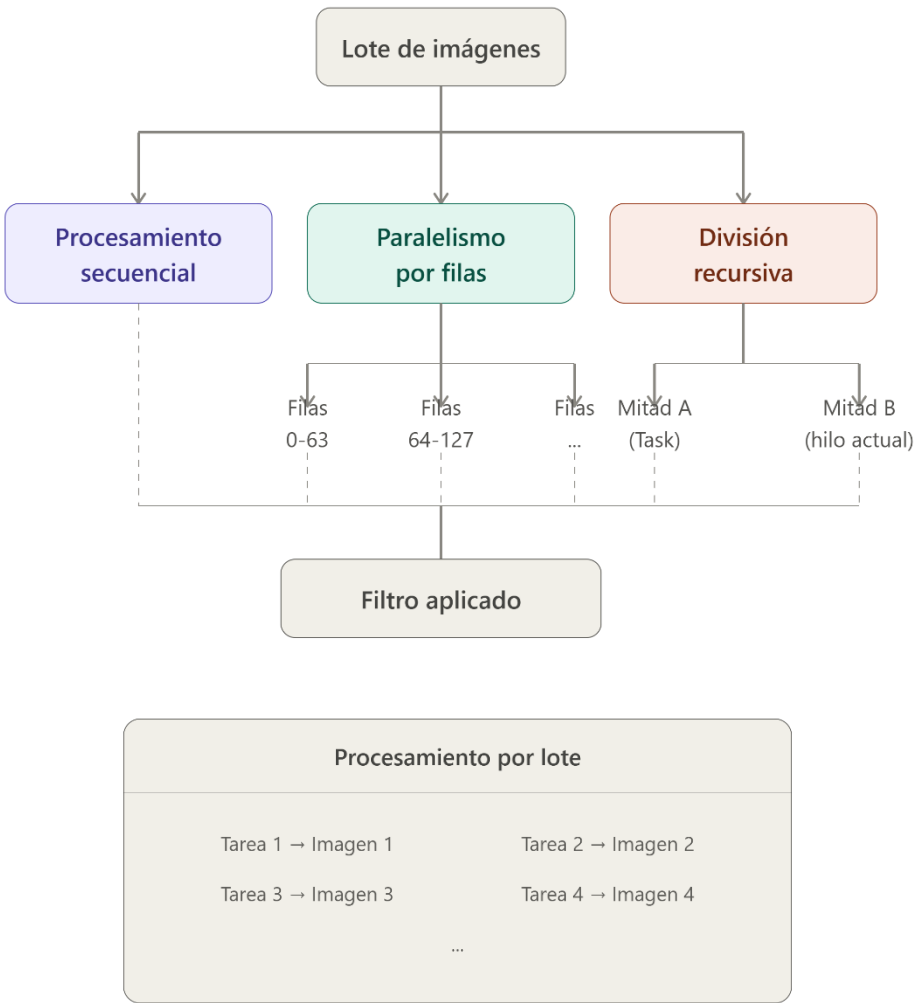


Figura 2. Distribución del trabajo entre las estrategias de procesamiento.

En el modo secuencial, las imágenes y sus píxeles son procesados uno después de otro utilizando un solo flujo de ejecución. Aunque este método no aprovecha varios núcleos simultáneamente, proporciona el tiempo de referencia necesario para calcular la aceleración obtenida por los métodos paralelos.

En el procesamiento paralelo por filas, una imagen se divide en filas y estas se distribuyen mediante `Parallel.For`. Como cada iteración trabaja sobre una fila diferente, varios hilos pueden aplicar el filtro simultáneamente sin modificar los mismos píxeles.

En la división recursiva, el conjunto de filas se divide repetidamente en dos regiones. Una mitad se envía a una tarea, mientras que la otra continúa en el hilo actual. Cada región sigue dividiéndose hasta alcanzar un tamaño mínimo de 64 filas o la profundidad máxima permitida. Cuando una región cumple una de estas condiciones, comienza inmediatamente a aplicar el filtro, aunque otras regiones todavía se estén dividiendo.

Por ejemplo, para una imagen de 500 filas, las regiones se dividen aproximadamente de la siguiente manera:

```
500 filas
├── 250 filas
│   ├── 125 filas
│   │   ├── 62 filas → aplicar filtro
│   │   └── 63 filas → aplicar filtro
│   └── 125 filas
│       ├── 62 filas → aplicar filtro
│       └── 63 filas → aplicar filtro
└── 250 filas
    ├── 125 filas
    │   ├── 62 filas → aplicar filtro
    │   └── 63 filas → aplicar filtro
    └── 125 filas
        ├── 62 filas → aplicar filtro
        └── 63 filas → aplicar filtro
```


Este ejemplo produce ocho regiones finales y siete tareas creadas explícitamente. Sin embargo, esto no significa que necesariamente se creen ocho hilos diferentes, debido a que las tareas son distribuidas por el `ThreadPool` de .NET entre los hilos disponibles.

En el **procesamiento paralelo por lotes**, cada imagen completa representa una unidad de trabajo independiente. `Parallel.ForEach` distribuye las imágenes entre los procesadores lógicos disponibles. De esta manera, un hilo puede procesar una imagen mientras otros hilos procesan imágenes diferentes. Esta estrategia resulta especialmente conveniente cuando el directorio de entrada contiene una gran cantidad de archivos.

4.3 Estrategia de paralelización utilizadas

La solución utiliza paralelismo de datos, ya que una misma operación se aplica simultáneamente sobre diferentes partes de los datos. Dependiendo del modo seleccionado, los datos pueden ser las filas de una misma imagen, regiones obtenidas recursivamente o varias imágenes independientes.

La estrategia paralela por filas utiliza `Parallel.For` para distribuir las filas de una imagen. Cada iteración recibe el índice de una fila y aplica el filtro sobre todos sus píxeles. Como las filas no se superponen, los hilos pueden trabajar concurrentemente sin necesidad de bloquear cada operación sobre los píxeles.

La estrategia recursiva utiliza objetos `Task` para aplicar el principio de divide y vencerás. Cada región grande se divide en dos mitades: una se procesa mediante una tarea del `ThreadPool` y la otra en el hilo actual. El programa utiliza mecanismos de espera para garantizar que ambas mitades hayan terminado antes de considerar completada la imagen.

La estrategia por lotes utiliza `Parallel.ForEach` para procesar varias imágenes simultáneamente. El grado máximo de paralelismo se configura tomando como referencia `Environment.ProcessorCount`, que devuelve la cantidad de procesadores lógicos disponibles en el equipo.

Debido a que varias tareas pueden intentar modificar las métricas compartidas al mismo tiempo, fue necesario utilizar mecanismos de sincronización. `Interlocked` permite actualizar los contadores de imágenes y píxeles de manera atómica. `lock` protege las secciones donde varias tareas podrían escribir simultáneamente en la consola.

`ConcurrentBag` permite almacenar de forma segura los errores producidos durante el procesamiento paralelo.

Una vez finalizado el procesamiento, se comparan las estrategias utilizando tres indicadores principales. El tiempo total representa la duración completa de la operación. El speedup indica cuántas veces una estrategia es más rápida que la ejecución secuencial:

$$\text{Speedup} = \text{tiempo secuencial} / \text{tiempo de la estrategia}$$

La eficiencia indica qué proporción de la capacidad teórica de los procesadores fue aprovechada:

$$\text{Eficiencia} = (\text{speedup} / \text{procesadores lógicos}) \times 100$$

Los resultados pueden variar según la cantidad de imágenes, sus dimensiones, el filtro seleccionado, la carga del equipo y las operaciones de lectura y escritura realizadas en el almacenamiento.

4.4 Herramientas y tecnologías empleadas

El proyecto fue desarrollado utilizando **C#** como lenguaje de programación y **.NET 8** como plataforma de ejecución. **C#** proporciona las estructuras necesarias para trabajar con programación orientada a objetos, administración de archivos, medición de rendimiento, tareas y procesamiento paralelo.

Para implementar el paralelismo se utilizó la **Task Parallel Library (TPL)**, integrada en .NET. Esta biblioteca proporciona clases y métodos de alto nivel como `Task`, `Parallel.For` y `Parallel.ForEach`, permitiendo distribuir el trabajo sin administrar manualmente la creación y destrucción de cada hilo.

El `ThreadPool` de .NET administra los hilos utilizados por las tareas. Esto evita crear un hilo nuevo para cada operación y permite reutilizar los hilos existentes de acuerdo con la capacidad y disponibilidad del sistema.

Para la manipulación de imágenes se utilizó la biblioteca **SixLabors.ImageSharp**. Esta herramienta permite cargar, modificar y guardar imágenes en formatos como JPG, JPEG, PNG y BMP. También facilita el acceso a los píxeles y la aplicación de transformaciones como escala de grises, inversión, brillo, contraste y desenfoque.

La clase **Stopwatch**, disponible en **System.Diagnostics**, fue utilizada para medir el tiempo empleado por cada estrategia. Las métricas obtenidas permiten calcular el rendimiento, el speedup y la eficiencia.

El desarrollo y las pruebas se realizaron en **Visual Studio 2022**, mientras que **Git** y **GitHub** fueron utilizados para el control de versiones y el trabajo colaborativo. Cada incremento fue desarrollado en una rama independiente, revisado mediante un pull request y posteriormente integrado en la rama principal utilizando **Squash and merge**.

En conjunto, estas herramientas permitieron desarrollar una solución modular capaz de procesar imágenes, distribuir el trabajo entre varios procesadores lógicos y comparar objetivamente el rendimiento de diferentes estrategias secuenciales y paralelas.

5. Implementación Técnica

5.1. Descripción de la estructura del proyecto:

Para facilitar la organización y mantenimiento del proyecto, este se encuentra dividido en varias carpetas, donde cada una agrupa componentes que cumple una función específica dentro del programa. Esta distribución permite separar las responsabilidades del programa y así evitar que toda la lógica se concentre en un único archivo o componente.

A su misma vez, este proyecto está dividido en diferentes módulos, los cuales son:

- ConsoleUI
- Filters
- Metrics
- Models
- Parallelism
- Processing

La carpeta **“ConsoleUI”** contiene los componentes relacionados con la interacción con el usuario mediante la consola. En esta sección se encuentra el menú del programa y el componente que se encarga de mostrar los resultados de las mediciones de rendimiento y

además de proporcionar una interfaz sencilla que permita al usuario seleccionar las operaciones que desea realizar y visualizar los resultados obtenidos posteriormente.

La carpeta ***“Filters”*** concentra la lógica que corresponde a los diferentes filtros que se le pueden aplicar a las imágenes. Entre ellos la conversión a escala de grises, inversión de colores, etc. Ya que al separar los filtros del resto del programa permite que cada operación de transformación pueda mantenerse y modificarse de manera independiente.

Luego, la carpeta ***“Models”*** contiene las estructuras utilizadas para mostrar la información dentro del programa. Entre estos elementos se encuentran los modelos pertenecientes a las rutas de los archivos, las mediciones de rendimiento y transportar la información entre las diferentes partes de la aplicación de manera organizada.

La carpeta ***“Processing”***, representa una de las partes centrales del programa, ya que contiene las diferentes estrategias utilizadas para procesar las imágenes. En ella se encuentran implementaciones para el procesamiento secuencial, paralelo, procesamiento paralelo por lotes, procesamiento recursivo y el modo de comparación de rendimiento. Esta organización es importante ya que permite comparar las diferentes formas de ejecutar la misma tarea y determinar que estrategia resulta más conveniente dependiendo de las características del problema.

Luego tenemos la carpeta ***“Parallelism”***, que contiene la lógica relacionada con la estrategia recursiva de paralelización. En ella se encuentra ***“RecursiveRowPartitioner”***, cuyo objetivo es dividir las filas de una imagen en diferentes segmentos para luego procesarlas de manera concurrente. Esta clase constituye una pieza importante del proyecto porque permite aplicar dicha estrategia para distribuir el trabajo.

Y finalmente, la carpeta ***“Metrics”*** contiene la lógica relacionada con la medición del rendimiento. Esta sección permite obtener información que luego puede utilizarse para comparar el comportamiento del procesamiento secuencial frente al paralelo.

5.2. Explicación del código clave:

El código clave del proyecto se encuentra principalmente en los componentes responsables de dividir y ejecutar el procesamiento de las imágenes. Uno de los componentes más importantes es ***“RecursiveRowPartitioner”***, debido a que implementa la estrategia recursiva utilizada para dividir el trabajo. La idea consiste en tratar una imagen como un conjunto de filas que pueden ser separadas en diferentes segmentos. En lugar de procesar toda la imagen como una única unidad, el algoritmo divide progresivamente el área del trabajo en partes más pequeñas.

Por ejemplo, si una imagen contiene una determinada cantidad de filas, el algoritmo puede dividirla inicialmente en dos regiones. Posteriormente, cada región puede volver a dividirse hasta alcanzar un tamaño considerado apropiado para ser procesado. De esta forma, el problema original se va transformando en varios problemas más pequeños que pueden ejecutarse de manera paralela.

La estrategia también considera la cantidad de procesadores lógicos disponibles mediante ***“Environment.ProcessorCount”***, de esta manera, la profundidad del particionamiento puede adaptarse a las capacidades del equipo donde se ejecuta el programa. Esto evita depender de una cantidad fija de tareas que podría no ser adecuada para todos los sistemas.

Otro elemento importante es ***“RecursiveImageProcessor”***, que utiliza la división realizada por ***“RecursiveRowPartitioner”*** para ejecutar realmente las operaciones sobre la imagen. Una vez que se obtiene una región determinada, el programa recorre sus filas y posteriormente los píxeles correspondientes. Para cada píxel se aplica la transformación al filtro seleccionado.

Los filtros independientes, como escala de grises, inversión de colores, brillo y contraste, se pueden procesar de forma relativamente sencilla porque la transformación de un píxel no depende directamente del resultado obtenido en otro píxel. Esto permite dividir la imagen en regiones y procesarlas de manera simultánea sin que una tarea tenga que esperar a otra.

El desenfoque presenta una situación diferente, debido a que para calcular el nuevo valor de un píxel es necesario consultar los píxeles que se encuentran alrededor de él. Por esta

razón, el proyecto utiliza una copia de la imagen original como fuente de información y escribe los resultados en otra imagen.

En el caso de *“ParallelImageProcessor”* , permite aprovechar varios núcleos del procesador para realizar de manera simultánea diferentes partes de la misma imagen.

Por otra parte, *“ParallelBatchImageProcessor”* aplica el paralelismo a un nivel diferente, es decir, permite procesar varias imágenes del conjunto de entrada de forma concurrente. Para ello utiliza mecanismos como *“Parallel.ForEach”*, limitando el grado de paralelismo de acuerdo con los recursos disponibles.

El programa también mantiene una implantación secuencial, mediante *“SequentialImageProcessor”*. Esta implementación es especialmente importante porque permite establecer una referencia con la que comparar las estrategias paralelas. De esta manera, el proyecto no solamente demuestra que es posible utilizar el paralelismo, sino que también permite analizar si su utilización produce realmente una mejora en el tiempo de ejecución.

5.3. Uso de mecanismos de sincronización:

Debido a que el programa ejecuta múltiples tareas de manera concurrente, es necesario utilizar mecanismos de sincronización que permitan controlar el acceso a los recursos compartidos. Sin estas medidas podrían producirse problemas como condiciones de carrera, pérdida de información o resultados inconsistentes.

Uno de los mecanismos utilizados es Interlocked. Este mecanismo se utiliza principalmente para modificar de manera segura contadores compartidos entre diferentes tareas. Por ejemplo, durante el procesamiento paralelo puede ser necesario contabilizar cuantas imágenes han sido procesadas correctamente, cuantas han procesado errores o cuantos píxeles han sido procesados.

Si varias tareas intentaran modificar simultáneamente una variable utilizando simplemente una operación como “contador++”, podrían producirse resultados incorrectos. Esto sucede porque dicha operación aparentemente sencilla implica varias operaciones internas y puede ser interrumpida por otro hilo.

Mediante `Interlocked.Increment` o `Interlocked.Add`, la modificación se realiza de forma atómica. Esto significa que la operación se completa como una unidad y evita que dos tareas interfieran incorrectamente entre sí.

Otro mecanismo empleado es `ConcurrentBag`, utilizado para almacenar información compartida, particularmente los errores producidos durante el procesamiento. Una colección tradicional como `List<T>` no está diseñada para que varios hilos agreguen elementos simultáneamente sin protección adicional. `ConcurrentBag`, en cambio, está diseñada específicamente para escenarios concurrentes.

Por lo tanto, diferentes tareas pueden detectar errores y agregarlos a la colección sin necesidad de implementar manualmente mecanismos complejos de bloqueo.

También se utiliza el mecanismo **lock**, principalmente para controlar el acceso a la consola. La escritura en consola constituye un recurso compartido y, si varias tareas intentaran escribir simultáneamente, los mensajes podrían aparecer mezclados.

El uso de **lock** permite establecer una sección crítica en la cual solamente una tarea puede realizar la operación de escritura a la vez. De esta manera, se mantiene organizada y legible la información mostrada al usuario.

Además de estos mecanismos, el proyecto utiliza sincronización relacionada con la finalización de tareas. En la estrategia recursiva, una región de la imagen puede ser procesada mediante una tarea mientras otra región continúa siendo procesada por el flujo actual. Posteriormente, el programa espera a que la tarea creada termine antes de continuar con determinadas operaciones.

Esto es importante porque el procesamiento paralelo no significa simplemente ejecutar tareas y olvidarse de ellas. En determinados momentos es necesario asegurarse de que todas las partes del trabajo hayan finalizado antes de considerar completa una operación.

5.4. Justificación técnica de las decisiones tomadas:

Las decisiones técnicas utilizadas en el proyecto responden principalmente a la necesidad de demostrar de forma práctica los beneficios y desafíos de la programación paralela aplicada al procesamiento de imágenes.

Una de las primeras decisiones importantes fue separar el procesamiento en diferentes componentes. En lugar de colocar todos los filtros, algoritmos de paralelización, mediciones y operaciones de consola dentro de un único archivo, cada responsabilidad fue distribuida entre diferentes clases y carpetas. Esto facilita el mantenimiento del proyecto y permite modificar una determinada funcionalidad sin afectar innecesariamente al resto del sistema.

La decisión de utilizar filas como unidad de división de la imagen también tiene una justificación técnica. Una imagen está compuesta por una matriz de píxeles y puede dividirse fácilmente en regiones horizontales. De esta forma, diferentes tareas pueden encargarse de diferentes grupos de filas.

Esta estrategia resulta especialmente apropiada para filtros como escala de grises, inversión de colores, brillo y contraste, ya que cada píxel puede transformarse de manera independiente.

La utilización de una **estrategia recursiva** permite que la división del trabajo sea dinámica. En lugar de definir manualmente una cantidad determinada de regiones, el algoritmo comienza con una región grande y la divide progresivamente hasta alcanzar las condiciones establecidas.

Esto permite adaptar el procesamiento al tamaño de la imagen y evita tener que establecer una estructura rígida para todos los casos.

Sin embargo, se tomó la decisión de establecer un límite mínimo de filas por región. Esta decisión es necesaria porque un paralelismo excesivo puede ser contraproducente. Si cada pocas filas se convirtieran en una nueva tarea, el costo de crear, administrar y sincronizar esas tareas podría superar el beneficio obtenido mediante la ejecución paralela.

Por esa razón, el algoritmo busca un equilibrio entre el tamaño de las regiones y la cantidad de tareas.

También se decidió utilizar `Environment.ProcessorCount` como referencia para determinar el nivel de paralelismo. Esto permite que el programa tenga un comportamiento más adaptable a diferentes equipos. Una computadora con pocos procesadores no debería intentar ejecutar la misma cantidad de trabajo paralelo que una máquina con una cantidad considerablemente mayor de recursos.

Otra decisión importante fue mantener una versión secuencial del procesamiento. Desde una perspectiva académica, esto es fundamental porque permite comparar el rendimiento de ambas estrategias.

Si solamente existiera el procesamiento paralelo, sería difícil determinar cuánto beneficio real aporta la paralelización. Al disponer de una versión secuencial, es posible medir el tiempo necesario para procesar una imagen de forma tradicional y compararlo con el tiempo obtenido mediante las estrategias paralelas.

Esto también permite estudiar conceptos como aceleración, eficiencia y escalabilidad.

En cuanto a la sincronización, el uso de `Interlocked`, `ConcurrentBag` y `lock` responde a diferentes necesidades. No sería técnicamente adecuado utilizar el mismo mecanismo para todos los recursos compartidos. Por ello, cada herramienta se utiliza según el tipo de problema que debe resolver.

Por ejemplo, `Interlocked` resulta apropiado para contadores simples, mientras que `ConcurrentBag` es más conveniente para almacenar colecciones de datos producidos por múltiples tareas. En el caso de la consola, `lock` resulta adecuado porque se necesita controlar temporalmente el acceso a un recurso compartido.

Finalmente, el tratamiento especial del filtro de desenfoque demuestra una consideración importante relacionada con la dependencia de datos. Mientras que algunos filtros pueden procesar cada píxel de forma independiente, el desenfoque necesita consultar píxeles vecinos. Por ello, utilizar una copia de los datos originales permite que diferentes tareas consulten información consistente mientras escriben sus resultados sin modificar la fuente que otras tareas necesitan.

En consecuencia, las decisiones tomadas no buscan únicamente hacer que el programa "funcione en paralelo", sino solucionar los diferentes problemas que aparecen cuando un algoritmo tradicional se adapta a un entorno concurrente.

6. Evaluación de Desempeño

6.1. Metodología de la comparativa

Para evaluar el desempeño de las diferentes estrategias de paralelización implementadas, se utilizó el modo "Comparar los modos" del programa, el cual ejecuta de manera consecutiva las cuatro estrategias disponibles (Secuencial, Paralelo por filas, División recursiva y Lote paralelo) sobre exactamente el mismo conjunto de imágenes y el mismo filtro, garantizando que las comparaciones sean equivalentes entre sí.

Las pruebas se realizaron utilizando el filtro de Inversión de colores como caso base, debido a que representa una transformación de bajo costo computacional por píxel, lo que permite evaluar el comportamiento de las distintas estrategias bajo una carga de procesamiento mínima por imagen. El equipo utilizado para las pruebas cuenta con 8 procesadores lógicos disponibles (Environment.ProcessorCount = 8), dato reportado automáticamente por el programa al inicio de cada comparación.

Con el propósito de evaluar la escalabilidad de la solución conforme aumenta el volumen de datos procesados, se realizaron cinco ejecuciones independientes, variando la cantidad de imágenes entre 150 y 1950. Las imágenes utilizadas fueron generadas mediante un servicio externo de fotografías reales, distribuidas entre distintos tamaños y resoluciones.

6.2. Comparativa entre ejecución secuencial y paralela

La siguiente tabla resume el *tiempo total* obtenido por cada estrategia en cada una de las cinco ejecuciones realizadas:

Imágenes	Secuencial (ms)	Paralelo por filas (ms)	División recursiva (ms)	Lote paralelo (ms)
150	5,311.22	2,928.79	3,347.01	1,215.32
250	8,764.48	5,096.01	5,727.18	2,165.40
500	17,325.92	10,777.91	12,887.04	4,978.61

Imágenes	Secuencial (ms)	Paralelo por filas (ms)	División recursiva (ms)	Lote paralelo (ms)
1350	66,573.87	40,931.68	52,345.24	14,709.52
1950	70,459.10	56,842.18	60,386.87	24,012.02

En todas estas ejecuciones, el Lote paralelo se mantuvo como la estrategia más rápida, seguida por el Paralelo por filas, mientras que la División recursiva ocupó consistentemente el último lugar entre las estrategias paralelas. La diferencia entre el modo Secuencial y el Lote paralelo alcanzó su punto más alto en la ejecución de 1350 imágenes, donde el tiempo total se redujo en un 77.9% (de 66,573.87 ms a 14,709.52 ms).

6.3. Métricas: tiempo de ejecución, eficiencia, escalabilidad

Antes de presentar estos resultados se debe tomar en cuenta que la estrategia Secuencial no se incluye en esta tabla, debido a que ésta funciona como referencia base para el cálculo de las demás. Por definición, su Speedup y Eficiencia son siempre 1.00x y 100% respectivamente, ya que el tiempo de las demás estrategias se calcula en función del tiempo que ella tomó. Al no variar entre ejecución y ejecución, no aporta información adicional para el análisis de escalabilidad.

La siguiente tabla presenta el Speedup y la Eficiencia obtenidos por las tres estrategias paralelas en cada ejecución:

Imágenes	Paralelo por filas		División Recursiva		Lote Paralelo	
	Speedup	Eficiencia	Speedup	Eficiencia	Speedup	Eficiencia
150	1.81x	22.67%	1.59x	19.84%	4.37x	54.63%
250	1.72x	21.50%	1.53x	19.13%	4.05x	50.59%

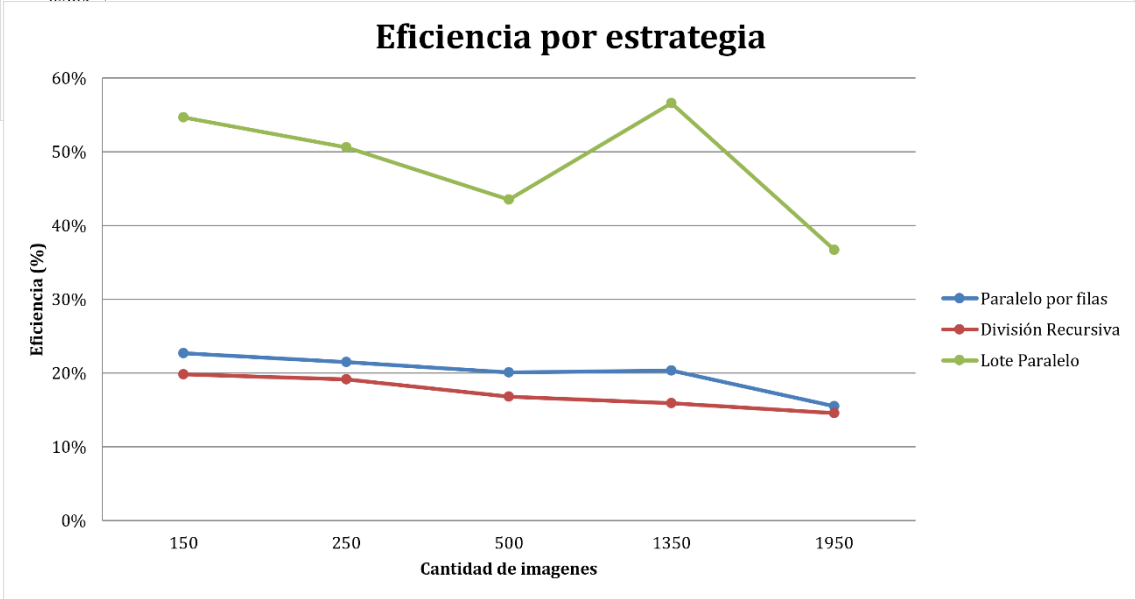
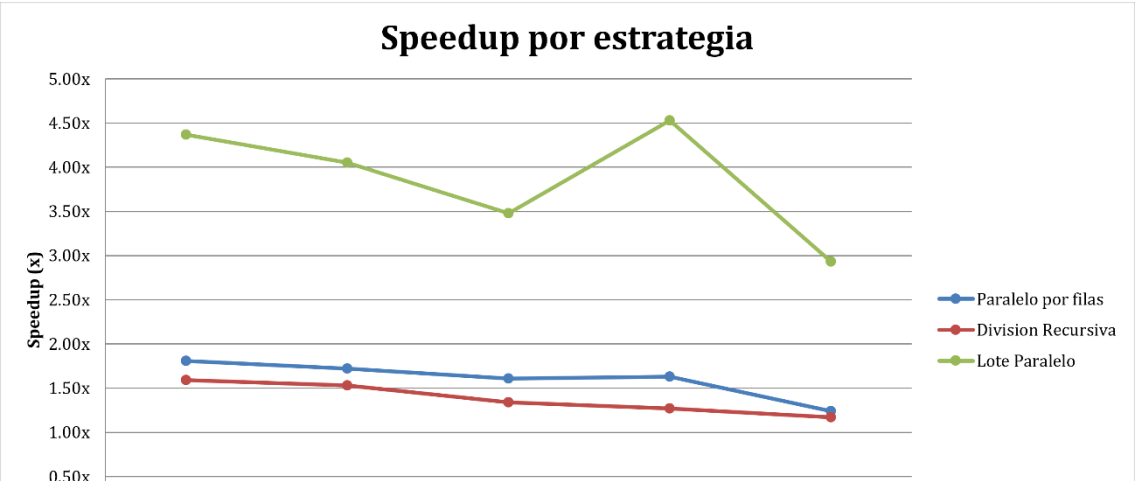
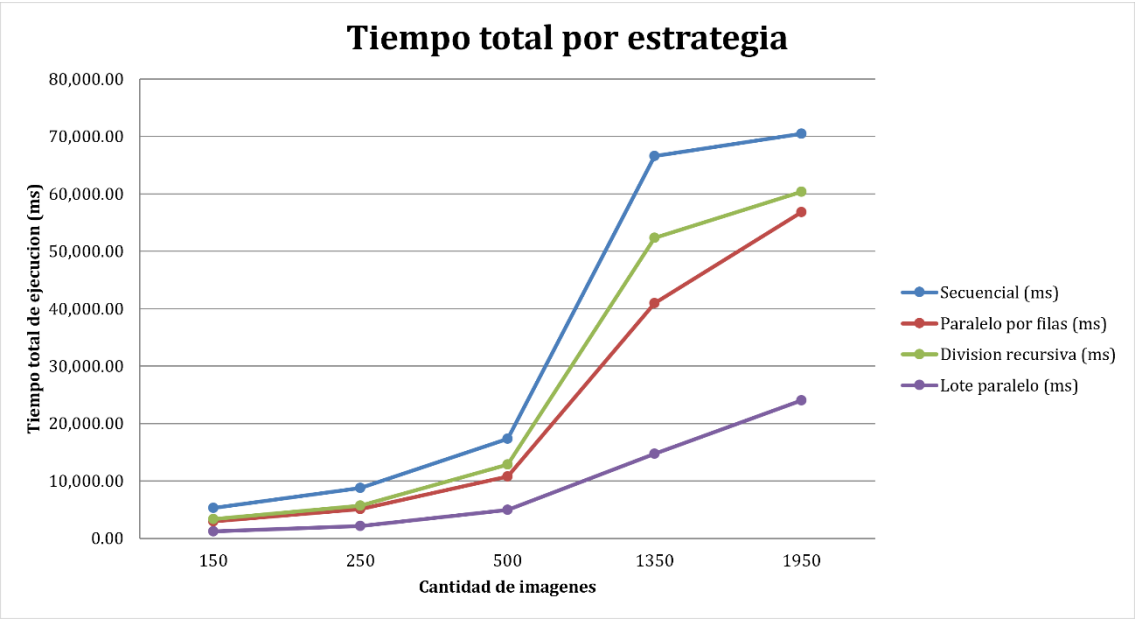
Imágenes	Paralelo por filas		División Recursiva		Lote Paralelo	
	Speedup	Eficiencia	Speedup	Eficiencia	Speedup	Eficiencia
500	1.61x	20.09%	1.34x	16.81%	3.48x	43.50%
1350	1.63x	20.33%	1.27x	15.90%	4.53x	56.57%
1950	1.24x	15.49%	1.17x	14.58%	2.93x	36.68%

En las primeras tres ejecuciones (150 a 500 imágenes) se observa una tendencia decreciente clara en el Speedup de las tres estrategias paralelas conforme crece el volumen: el Paralelo por filas pasa de 1.81x a 1.61x, la División recursiva de 1.59x a 1.34x, y el Lote paralelo de 4.37x a 3.48x. Esto tiene sentido según la Ley de Amdahl: mientras más imágenes hay que procesar, la parte que no se puede paralelizar (leer y guardar cada imagen en disco) ocupa más tiempo del total, y eso deja menos espacio para que el paralelismo siga mejorando el rendimiento.

En la ejecución de 1350 imágenes se observa una recuperación puntual en el Lote paralelo, que alcanza su mejor resultado de toda la evaluación (4.53x de speedup, 56.57% de eficiencia), mientras que el Paralelo por filas y la División recursiva se mantienen en niveles similares a la ejecución anterior. Sin embargo, en la ejecución de mayor volumen (1950 imágenes) las tres estrategias registran sus valores más bajos de speedup y eficiencia, con el Lote paralelo cayendo a 2.93x (36.68%), el Paralelo por filas a 1.24x (15.49%) y la División recursiva a 1.17x (14.58%).

Es importante señalar que el tiempo total de la ejecución de 1950 imágenes no escaló de forma estrictamente proporcional respecto a la de 1350: a pesar de procesar un 44% más de imágenes, el tiempo secuencial solo aumentó un 5.8%. Esto responde a la variabilidad propia de medir rendimiento en un equipo de uso general, sin que esto reste validez a la tendencia general identificada.

6.4. Gráficas con resultados



6.5. Análisis de cuellos de botella o limitaciones

Los resultados obtenidos revelan dos limitaciones distintas:

1. El filtro utilizado limita el beneficio del paralelismo por píxel. La inversión de colores es una operación de bajo costo computacional, por lo que el tiempo de cada imagen está dominado por la lectura y escritura en disco, no por el cálculo sobre los píxeles. El Paralelo por filas y la División recursiva únicamente paralelizan el cálculo dentro de cada imagen, pero siguen cargando y guardando las imágenes una por una, lo que explica su eficiencia consistentemente baja (15% a 22%). El Lote paralelo, al distribuir también la I/O entre varias imágenes simultáneamente, obtiene un beneficio mucho mayor sin depender del costo del filtro.
2. A partir de cierto volumen, el sistema se ve afectado por saturación de recursos. La caída sostenida en los píxeles procesados por segundo del modo Secuencial en las corridas de 1350 y 1950 imágenes, junto con el speedup de la División recursiva por debajo de 1.0x, indica que el cuello de botella deja de ser la I/O de una imagen individual y pasa a ser la acumulación de operaciones de disco generadas por las cuatro estrategias ejecutándose consecutivamente. Este efecto perjudica más a las estrategias que tardan más en completarse (Secuencial y Recursiva), mientras que el Lote paralelo, al terminar en una fracción del tiempo, resulta menos afectado y por eso mejora su eficiencia en las corridas más grandes.

Otras limitaciones identificadas:

- El límite mínimo de 64 filas por región en la estrategia recursiva puede no ser óptimo para imágenes de baja resolución.
- El lock usado en la escritura de consola del Lote paralelo introduce un punto de serialización, aunque de impacto mínimo frente al beneficio de superponer la I/O.
- Sería recomendable complementar este análisis con el filtro de Desenfoque, computacionalmente más costoso, para verificar si las estrategias que paralelizan dentro de una imagen logran superar al Lote paralelo cuando el cómputo, y no la I/O, domina el tiempo total.

7. Trabajo en equipo:

7.1. Descripción del reparto de tareas:

Rusbel Ohara Uceta (Líder y QA): Encargado de supervisar y de administrar los procesos a realizar en este proyecto en cuestión, también asegurando de que todas las nuevas implementaciones técnicas o de documentación cumplan con los estándares de calidad esperados por el maestro, además de poseer al igual que mis demás compañeros el conocimiento técnico necesario para llevar a cabo este proyecto.

Richard Torres (Desarrollador): Encargado de desarrollar la solución tecnológica en base a la propuesta presentada al maestro, asegurando de aplicar buenas prácticas tanto en los métodos de desarrollo como en el código posteriormente presentado ante el maestro, y se es necesario mejorar o corregir aspectos del código según se requiera.

Ismarlin Martinez (Encargada de las métricas comparativos): Encargada de llevar a cabo pruebas de comparación de los diferentes modos del programa para poder determinar qué modo en el procesamiento de imágenes es el más óptimo para “X” situación aplicando los filtros necesarios y/o pruebas para conseguir dicha respuesta.

7.2. Herramientas utilizadas para coordinación:

En este caso decidimos utilizar Git o GitHub para un mayor control en lo que a cambios durante el desarrollo del programa se refiere, además de la gran comodidad que nos otorga su extensión de escritorio para así poder tener mayor visión o control sobre dichos cambios, además de las facilidades que nos dieron o nos dio la plataforma para realizarlos. Sin mencionar que al utilizar la versión de escritorio de GitHub nos permitió clonar de mejor manera el repositorio así estar al pendiente o siempre actualizados de cada una de las nuevas implementaciones, ya sea técnicas o de documentación, en nuestro trabajo.

8. Conclusiones

Principales aprendizajes técnicos:

Los principales aprendizajes que conseguimos a partir de este proyecto fue la capacidad de poder dividir un trabajo en diferentes segmentos para así conseguir una mayor eficiencia. ¿A qué me refiero con esto? Durante este trabajo nos hemos visto envueltos en un problema, y es que, al ser un programa relacionado con procesamiento de imágenes, al principio se pensaba que se podía hacer todo en una misma clase, lo cual era un pensamiento erróneo.

Lo que nos llevó a dividir el trabajo en varios módulos, cada grupo con su funcionalidad correspondiente, lo cual mejoró bastante la eficiencia y a su misma vez aplicamos la estrategia que se nos asignó, haciendo así o consiguiendo así un programa funcional, escalable y sobre todo, amigable con el usuario.

Retos enfrentados y superados

Durante el desarrollo de este proyecto nos encontramos con varios retos, siendo uno de los principales la interpretación correcta de los resultados obtenidos en las pruebas de rendimiento. En un inicio, algunos de los datos que arrojaba el programa nos parecían contradictorios, por ejemplo, hubo corridas donde una estrategia paralela se comportaba de manera distinta a lo que esperábamos según el volumen de imágenes procesadas. Esto nos llevó a entender que no toda variación en los resultados representa un error en el código, sino que también puede deberse a factores externos como el estado del equipo en el momento de la prueba. Aprendimos así a diferenciar entre una tendencia real de comportamiento y el ruido natural que se produce al medir el rendimiento en un entorno no controlado.

Otro reto importante fue garantizar que las comparaciones entre las diferentes estrategias fueran justas y equivalentes. Para esto, tuvimos que asegurarnos de que las cuatro estrategias se ejecutaran sobre exactamente el mismo conjunto de imágenes y en la misma sesión, evitando así que las diferencias en los resultados se debieran a condiciones distintas entre una prueba y otra. Asimismo, tuvimos que resolver desafíos propios de la

implementación, como el manejo del filtro de desenfoque, el cual, a diferencia de los demás filtros, requería trabajar con una copia de solo lectura de la imagen original para evitar que las tareas paralelas leyeran datos ya modificados por otras tareas.

Posibles mejoras o líneas futuras

A partir de los resultados obtenidos, identificamos varias oportunidades de mejora para versiones futuras del proyecto. En primer lugar, sería valioso repetir las pruebas de rendimiento utilizando un filtro computacionalmente más costoso, como el desenfoque, para contrastar estos resultados con los obtenidos mediante el filtro de inversión de colores, y así comprobar si las estrategias que paralelizan dentro de una misma imagen logran superar al procesamiento por lotes cuando el cálculo, y no la lectura o escritura en disco, se convierte en el factor dominante.

También sería interesante incorporar la posibilidad de controlar manualmente la cantidad de núcleos que utiliza el programa, con el fin de estudiar la escalabilidad respecto a los recursos de hardware de forma más directa, sin depender únicamente de variar el volumen de imágenes procesadas. Adicionalmente, dado que identificamos que la lectura y escritura en disco representa uno de los principales cuellos de botella del sistema, una mejora relevante sería explorar el uso de operaciones asíncronas para la carga y el guardado de imágenes. Finalmente, como línea futura más ambiciosa, se podría investigar el procesamiento mediante GPU, ya que este proyecto evidenció los límites que tiene el paralelismo aplicado únicamente sobre el CPU.

9. Referencias

Para el desarrollo del proyecto se consultaron fuentes relacionadas con el procesamiento digital de imágenes, los algoritmos recursivos y la programación paralela en C#.

Gonzalez y Woods presentan los fundamentos del procesamiento digital de imágenes, incluyendo la representación mediante píxeles, las transformaciones de intensidad y los filtros espaciales. Esta fuente sirve como base teórica para los filtros de escala de grises, brillo, contraste, inversión y desenfoque.

Cormen, Leiserson, Rivest y Stein explican el paradigma de divide y vencerás, utilizado como fundamento para el modo recursivo. En esta estrategia, el conjunto de filas de una imagen se divide repetidamente en regiones más pequeñas hasta alcanzar el tamaño mínimo establecido.

Sarcar aborda la programación paralela en C# y .NET utilizando tareas, ciclos paralelos, mecanismos de sincronización y colecciones concurrentes. Esta fuente se relaciona directamente con las herramientas utilizadas en el proyecto, como `Task`, `Parallel.For`, `Parallel.ForEach`, `Interlocked` y `lock`.

La documentación oficial de Microsoft sobre Task Parallel Library fue utilizada para consultar el funcionamiento del paralelismo de datos, la distribución del trabajo y la administración de tareas mediante el `ThreadPool`.

Fuentes bibliográficas

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). Cambridge, MA: The MIT Press.
<https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital image processing* (4.^a ed.). New York, NY: Pearson. <https://www.mathworks.com/academia/books/digital-image-processing-gonzalez.html>
- Microsoft. (2025). *Biblioteca paralela de tareas (TPL)*. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/standard/parallel-programming/task-parallel-library-tpl>
- Sarcar, V. (2024). *Parallel programming with C# and .NET: Fundamentals of concurrency and asynchrony behind fast-paced applications*. Berkeley, CA: Apress. <https://learning.oreilly.com/library/view/parallel-programming-with/9798868804885/>

10. Anexos

10.1 Manual de ejecución del sistema

Para ejecutar el procesador paralelo de imágenes se requiere un equipo con Windows, .NET 8 y Visual Studio 2022. También es necesario contar con acceso al repositorio público del proyecto y disponer de imágenes en alguno de los formatos admitidos: JPG, JPEG, PNG o BMP.

10.1.1 Descarga y apertura del proyecto

El código fuente puede obtenerse desde el repositorio público de GitHub indicado al final de este apartado. El usuario puede clonar el repositorio mediante Git o descargarlo como archivo ZIP.

Para clonarlo se utiliza el siguiente comando:

```
git clone https://github.com/XxNightlayerxX/Filtro\_Imagenes.git
```

Después de descargar el proyecto, se debe abrir en Visual Studio 2022 el archivo de solución:

```
Filtro_Imagenes.sln
```

La solución contiene el proyecto de consola `ProcesadorImagenesParalelo`, organizado en componentes para la interfaz de consola, filtros, modelos, métricas, procesamiento de imágenes y estrategias de paralelización.

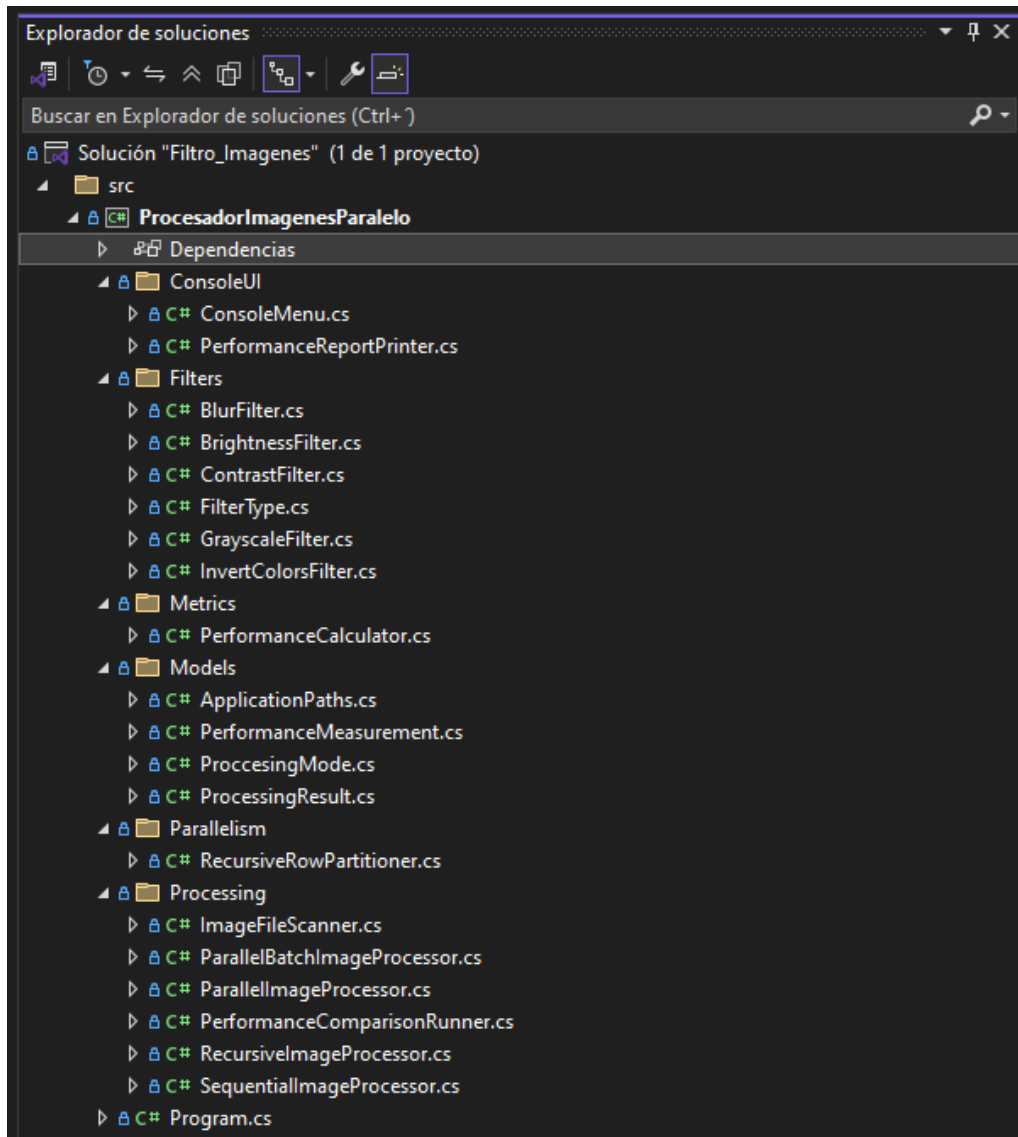


Figura 1. Estructura del proyecto abierta en el Explorador de soluciones de Visual Studio 2022.

10.1.2 Compilación inicial y creación de directorios

Una vez abierta la solución, se debe compilar el proyecto desde la opción Compilar solución de Visual Studio. También puede utilizarse la combinación de teclas:

Ctrl + Mayús + B

La aplicación utiliza como directorio base la carpeta de ejecución generada por .NET:

```
src\ProcesadorImagenesParalelo\bin\Debug\net8.0
```

Durante la ejecución se crean automáticamente los directorios requeridos:

Input

Output

- └─ Sequential
- └─ Parallel
- └─ Recursive
- └─ ParallelBatch

El directorio Input almacena las imágenes originales. Las carpetas contenidas en Output guardan los resultados de acuerdo con la estrategia seleccionada.

10.1.3 Preparación de las imágenes

Las imágenes que serán procesadas deben colocarse directamente en:

`src\ProcesadorImagenesParalelo\bin\Debug\net8.0\Input`

No deben permanecer dentro de una subcarpeta, ya que la aplicación examina únicamente el nivel principal de Input.

Para las pruebas del proyecto se utilizó un conjunto de 500 imágenes en formato JPG. La aplicación también admite archivos JPEG, PNG y BMP.

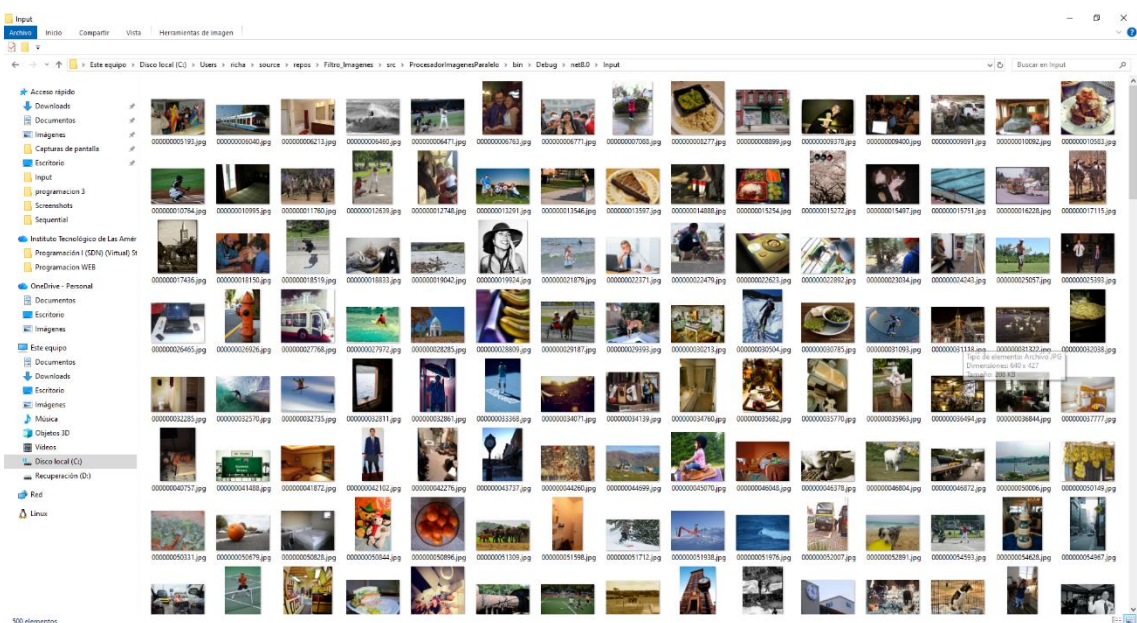


Figura 2. Directorio Input con las 500 imágenes utilizadas para las pruebas de rendimiento.

El paquete de prueba puede descargarse desde la sección **Releases** del repositorio. Después de descargar `imagenes-prueba-500.zip`, su contenido debe extraerse directamente en la carpeta Input.

10.1.4 Ejecución de la aplicación

Para ejecutar el programa desde Visual Studio se puede utilizar:

Ctrl + F5

También puede emplearse el botón de ejecución ubicado en la barra superior.

Al iniciar, la aplicación detecta las imágenes disponibles y presenta los filtros implementados:

1. Escala de grises
2. Inversión de colores
3. Brillo
4. Contraste
5. Desenfoque

Después solicita el modo de procesamiento:

1. Secuencial
2. Paralelo por filas
3. División recursiva
4. Lote paralelo
5. Comparar los modos

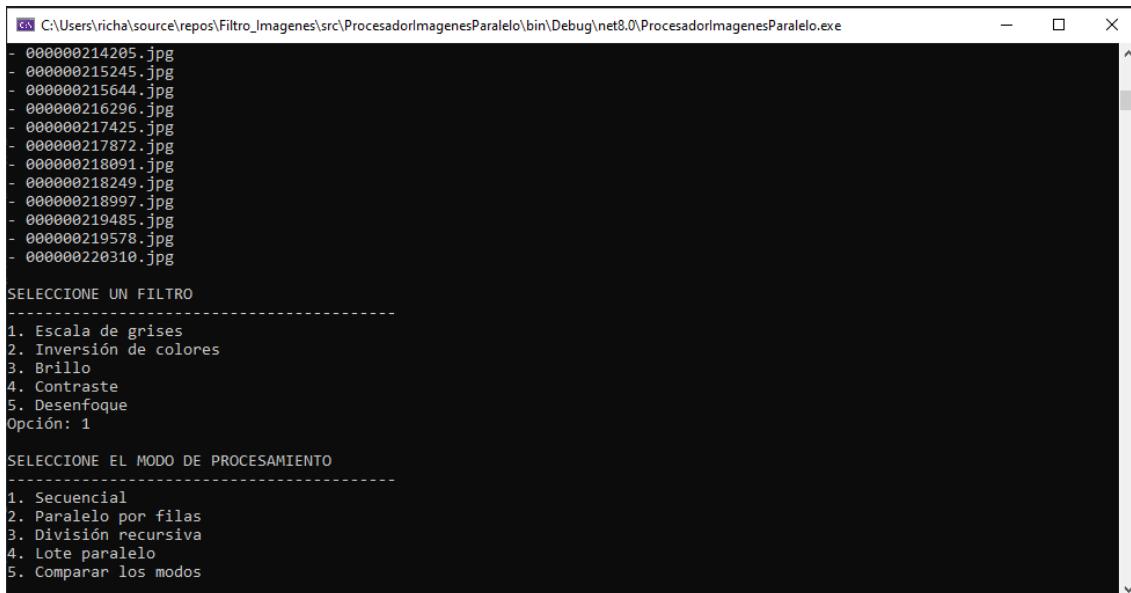


Figura 3. Menú para seleccionar el filtro y el modo de procesamiento.

Algunos filtros solicitan un valor adicional. El brillo y el contraste requieren un nivel de ajuste, mientras que el desenfoque utiliza un valor que determina la intensidad o radio del efecto.

10.1.5 Procesamiento y presentación de resultados

Una vez seleccionadas las opciones, el programa carga las imágenes, aplica el filtro y guarda los resultados en el directorio correspondiente.

Al finalizar, la consola presenta:

Cantidad de imágenes procesadas
Cantidad de imágenes fallidas
Total de píxeles procesados
Tiempo empleado por el filtro
Píxeles procesados por segundo
Directorio de salida

En la prueba paralela por filas se procesaron satisfactoriamente las 500 imágenes, sin archivos fallidos. El sistema procesó 138,412,502 píxeles en aproximadamente 566.09 milisegundos de tiempo acumulado del filtro, alcanzando una velocidad aproximada de 244,504,925 píxeles por segundo.

```
Consola de depuración de Microsoft Visual Studio

Procesando en paralelo: 00000218997.jpg
Completada en 1.00 ms

Procesando en paralelo: 00000219485.jpg
Completada en 0.88 ms

Procesando en paralelo: 00000219578.jpg
Completada en 0.91 ms

Procesando en paralelo: 00000220310.jpg
Completada en 0.54 ms

RESULTADOS DEL LOTE
-----
Imágenes procesadas: 500
Imágenes fallidas: 0
Píxeles procesados: 138,412,502
Tiempo del filtro: 566.09 ms
Píxeles por segundo: 244,504,925

Resultados guardados en:
C:\Users\richa\source\repos\Filtro_Imagenes\src\ProcesadorImagenesParalelo\bin\Debug\net8.0\Output\Parallel

Presiona cualquier tecla para finalizar...

C:\Users\richa\source\repos\Filtro_Imagenes\src\ProcesadorImagenesParalelo\bin\Debug\net8.0\ProcesadorImagenesParalelo.e
xe (proceso 7756) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

Figura 4. Resultado del procesamiento paralelo por filas sobre un conjunto de 500 imágenes.

10.1.6 Verificación de las imágenes generadas

Los resultados deben comprobarse en la carpeta asociada con el modo utilizado. Para la ejecución paralela por filas, la ubicación es:

```
src\ProcesadorImagenesParalelo\bin\Debug\net8.0\Output\Parallel
```

Las imágenes conservan sus nombres originales, pero contienen la transformación seleccionada. En la prueba mostrada se utilizó el filtro de escala de grises.

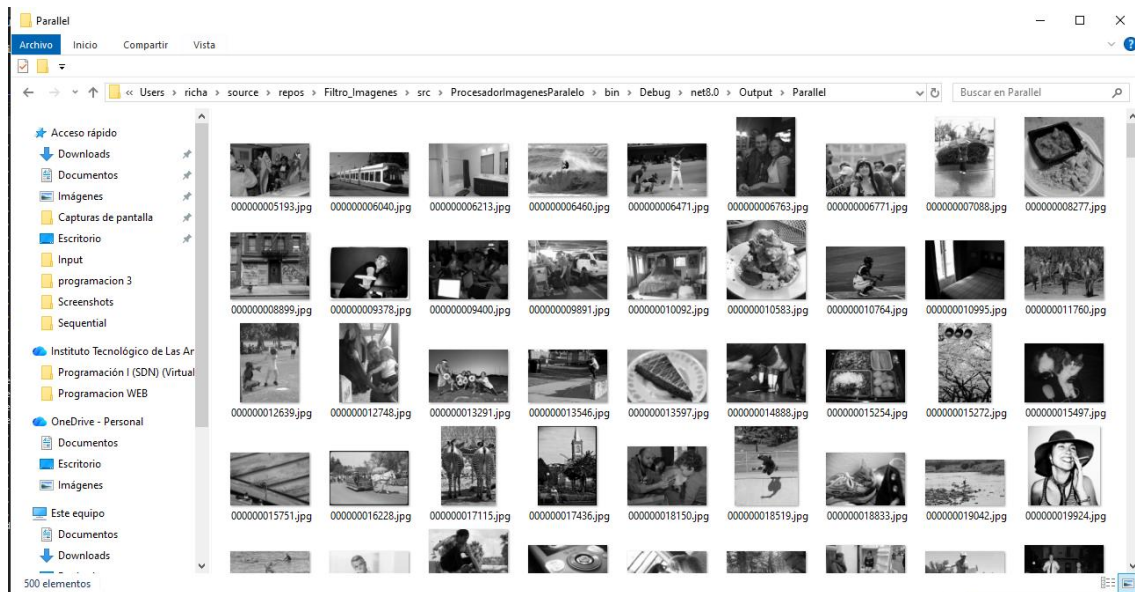


Figura 5. Imágenes transformadas a escala de grises y almacenadas en Output\Paralel.

10.2 Capturas adicionales y pruebas complementarias

10.2.1 Prueba con 500 imágenes

Para evaluar el funcionamiento del sistema bajo una carga significativa, se utilizó un conjunto de 500 imágenes. Durante la ejecución, la aplicación mostró el avance mediante un contador y el nombre del archivo procesado.

Esta prueba permitió comprobar que el sistema puede recorrer un lote completo, aplicar el filtro seleccionado, guardar los resultados y mantener el conteo correcto de imágenes procesadas.

```
Consola de depuración de Microsoft Visual Studio
Progreso: 475/500 - 000000043737.jpg
Progreso: 476/500 - 000000151938.jpg
Progreso: 477/500 - 000000096960.jpg
Progreso: 478/500 - 000000117425.jpg
Progreso: 479/500 - 000000022623.jpg
Progreso: 480/500 - 000000195045.jpg
Progreso: 482/500 - 000000080666.jpg
Progreso: 481/500 - 000000134689.jpg
Progreso: 483/500 - 000000165257.jpg
Progreso: 484/500 - 000000022892.jpg
Progreso: 485/500 - 000000219578.jpg
Progreso: 486/500 - 000000044260.jpg
Progreso: 487/500 - 000000097230.jpg
Progreso: 488/500 - 000000136633.jpg
Progreso: 489/500 - 000000220310.jpg
Progreso: 490/500 - 000000195165.jpg
Progreso: 491/500 - 000000152465.jpg
Progreso: 492/500 - 000000136772.jpg
Progreso: 493/500 - 000000097337.jpg
Progreso: 494/500 - 000000165336.jpg
Progreso: 495/500 - 000000153011.jpg
Progreso: 496/500 - 000000165351.jpg
Progreso: 497/500 - 000000165518.jpg
Progreso: 498/500 - 000000166165.jpg
Progreso: 499/500 - 000000166521.jpg
Progreso: 500/500 - 000000166768.jpg

RESULTADOS DE LA COMPARACIÓN
=====
Procesadores lógicos disponibles: 12
```

Figura 6. Progreso del procesamiento de las 500 imágenes utilizadas en la prueba comparativa.

La prueba finalizó con las 500 imágenes completadas y cero fallos, lo que demuestra que los formatos utilizados fueron procesados correctamente.

10.2.2 Comparación de las estrategias

La aplicación incluye un modo de comparación que ejecuta las cuatro estrategias utilizando las mismas imágenes y el mismo filtro:

- Procesamiento secuencial
- Procesamiento paralelo por filas
- División recursiva
- Procesamiento paralelo por lotes

Para cada estrategia se presentan el tiempo total, las imágenes procesadas, las imágenes fallidas, los píxeles por segundo, el speedup y la eficiencia.

En el equipo utilizado para la prueba se detectaron 12 procesadores lógicos. Los resultados obtenidos fueron los siguientes:

Estrategia	Tiempo total	Píxeles por segundo	Speedup	Eficiencia
Secuencial	7,399.63 ms	18,705,315	1.00x	100.00 %
Paralelo por filas	5,579.95 ms	24,805,330	1.33x	11.05 %
División recursiva	6,545.72 ms	21,145,493	1.13x	9.42 %
Lote paralelo	1,496.87 ms	92,467,698	4.94x	41.19 %

La estrategia más rápida fue el lote paralelo. Este resultado se debe a que las 500 imágenes constituyen unidades de trabajo independientes que pueden distribuirse entre los procesadores lógicos disponibles. Mientras un hilo procesa una imagen, otros pueden trabajar simultáneamente con archivos diferentes.

El speedup de 4.94x indica que, durante esta ejecución, el lote paralelo fue aproximadamente 4.94 veces más rápido que el procesamiento secuencial. La eficiencia de 41.19 % representa la aceleración obtenida en relación con los 12 procesadores lógicos disponibles.

```

Consola de depuración de Microsoft Visual Studio
Progreso: 497/500 - 000000216296.jpg
Progreso: 498/500 - 000000220310.jpg
Progreso: 499/500 - 000000180751.jpg
Progreso: 500/500 - 000000180792.jpg

RESULTADOS DE LA COMPARACIÓN
=====
Procesadores lógicos disponibles: 12

Modo: Secuencial
Tiempo total: 7399.63 ms
Imágenes procesadas: 500
Imágenes fallidas: 0
Píxeles por segundo: 18,705,315
Speedup: 1.00x
Eficiencia: 100.00%
-----
Modo: Paralelo por filas
Tiempo total: 5579.95 ms
Imágenes procesadas: 500
Imágenes fallidas: 0
Píxeles por segundo: 24,805,330
Speedup: 1.33x
Eficiencia: 11.05%
-----
Modo: División recursiva
Tiempo total: 6545.72 ms
Imágenes procesadas: 500
Imágenes fallidas: 0
Píxeles por segundo: 21,145,493
Speedup: 1.13x
Eficiencia: 9.42%
-----
Modo: Lote paralelo
Tiempo total: 1496.87 ms
Imágenes procesadas: 500
Imágenes fallidas: 0
Píxeles por segundo: 92,467,698
Speedup: 4.94x
Eficiencia: 41.19%
-----
Estrategia más rápida: Lote paralelo
Presiona cualquier tecla para finalizar...

```

Figura 7. Resultados de la comparación entre las cuatro estrategias de procesamiento.

Los resultados pueden variar según las características del equipo, la cantidad y resolución de las imágenes, el filtro seleccionado, las aplicaciones ejecutadas en segundo plano y la velocidad del almacenamiento. Por esta razón, estas mediciones representan los resultados obtenidos en el entorno específico de prueba

10.2.3 Validaciones realizadas

Durante las pruebas se verificaron los siguientes aspectos:

- La solución compila y se ejecuta correctamente en .NET 8.
- La aplicación detecta archivos JPG, JPEG, PNG y BMP.
- El menú permite seleccionar filtros y modos de procesamiento.
- Las 500 imágenes fueron procesadas sin fallos.
- Las imágenes resultantes fueron guardadas en el directorio correspondiente.
- Las estrategias producen las métricas necesarias para comparar su rendimiento.
- El modo de comparación identifica automáticamente la estrategia más rápida.
- La ejecución finaliza correctamente con código de salida cero.

10.3 Enlace al repositorio público de Git

El código fuente del proyecto se encuentra disponible públicamente en GitHub:

Repositorio:

https://github.com/XxNightlayerxX/Filtro_Imagenes

El repositorio contiene la solución desarrollada en C#, la implementación de los filtros, los procesadores secuenciales y paralelos, las clases de métricas y la documentación del proyecto.

La sección **Releases** incluye el paquete:

`imagenes-prueba-500.zip`

Este archivo permite descargar las 500 imágenes utilizadas en las pruebas. Después de extraerlas en el directorio Input, el profesor puede ejecutar el sistema y reproducir las comparaciones presentadas en este documento.

Para ejecutar el proyecto desde una copia local se puede utilizar:

```
git clone https://github.com/XxNightlayerxX/Filtro\_Imagenes.git
cd Filtro_Imagenes
dotnet restore
dotnet build
dotnet run --project
.\src\ProcesadorImagenesParalelo\ProcesadorImagenesParalelo.csproj
```

Después de la primera ejecución, deben colocarse las imágenes en la carpeta Input generada por la aplicación y volver a ejecutar el último comando.